

LABORATORIO DI INGEGNERIA DEI SISTEMI SOFTWARE

Introduction

Questo documento descrive il lavoro svolto nello **Sprint 0** per lo sviluppo del sistema **cargoservice**. L'obiettivo principale è analizzare, formalizzare e delineare il modello concettuale del sistema software che automatizza le operazioni di carico dei container nella stiva di una nave utilizzando un robot a guida differenziale (`cargorobot`).

Requirements

The company asks us to build service named **cargoservice** that should work as follows.

The `cargoservice` is able to receive a **request to load** a container sent by some customer by using the `pushbutton` of the `IOPort`.

- It sends the answer `retrylater`, if the **IOPort** is currently occupied by a container or if the system is **Out of service**
- It rejects the request when the hold is already full, i.e. the `slots1-4` are already occupied.
- Otherwise, it considers the system as **engaged**, detects a free slot and returns as answer the name of such a reserved slot. While engaged, the system must blink a Led.

When the **request to load** is accepted, the customer must move the container in the `sensor` area within prefixed amount of time (e.g. 30 secs), otherwise the systems becomes **disengaged**. Then, the `cargoservice` uses the `cargorobot` to move the container from the `IOPort` to the `slot5` (for marking the container) and then to the reserved slot.

The service must also show on the `display` on the `IOPort`:

- the current state of the `hold`
- the message '**Service working**', when it is all is going well
- the message '**Out of service**' if the `sonar sensor` measures a distance $D > D_{FREE}$ for at least 3 secs (perhaps a failure of the `sonar`).

Requirement analysis

Seguendo i requisiti, identifichiamo e formalizziamo le principali entità e interazioni del sistema.

Termini Chiave del Dominio (Nomi)

- **company:** L'azienda di spedizioni marittime che richiede l'automazione.
- **cargorobot:** Un robot a guida differenziale programmato per trasportare i container.
- **hold:** La stiva, un'area rettangolare piatta che contiene:
 - **slots1-4:** Aree di stoccaggio finali.
 - **slot5:** Area temporanea per la marcatura del container.
- **IOPort:** Punto di interazione per il cliente composto da un **pushbutton** e un **display**.
- **sensor (sonar):** Sensore associato alla IOPort per rilevare la presenza del container o eventuali guasti.
- **marker:** Dispositivo nello slot5 che etichetta il container e segnala il completamento.
- **Led:** Dispositivo luminoso indicatore dello stato del sistema.

Stati del Sistema ed Azioni Logiche

- **Validazione della richiesta di carico:** Invia `retrylater` se occupato/fuori servizio, rifiuta se pieno, altrimenti accetta (**engaged**), assegna uno slot e lampeggia il **Led**.
- **Posizionamento del container:** Entro 30 secondi, altrimenti timeout e sistema disimpegnato.
- **Sequenza di esecuzione:** `IOPort` → spostamento su `slot5` → attesa marcatura → spostamento su `slot riservato` → `HOME`.

Problem analysis

Analizziamo la natura dei componenti software e hardware e le modalità di comunicazione.

Natura dei Componenti

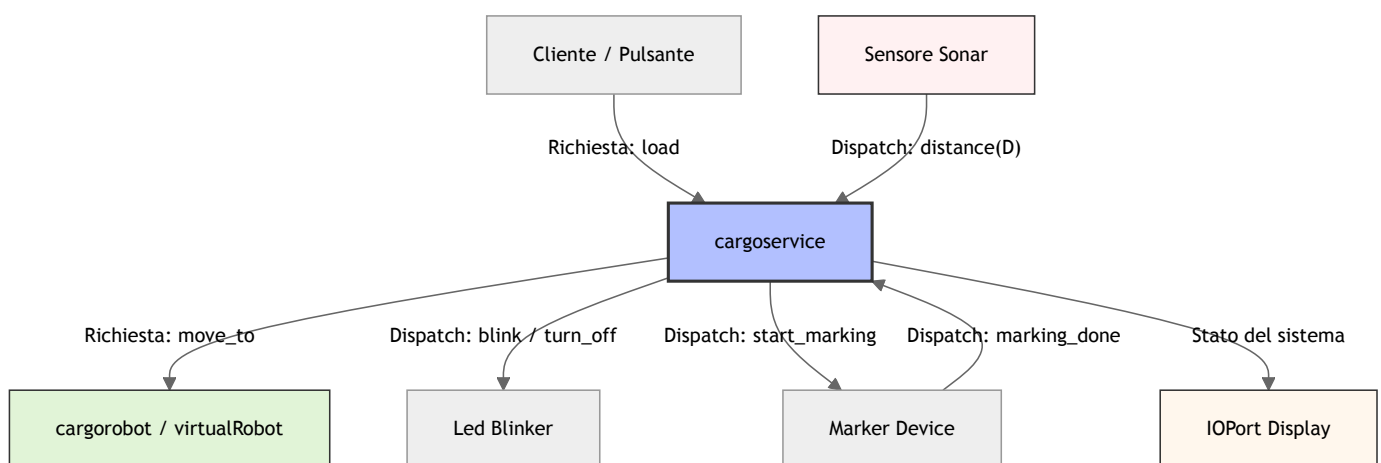
Componente	Tipo	Descrizione / Simulazione o Fisico
cargoservice	Attivo (Software)	Orchestratore centrale. Da sviluppare.

Componente	Tipo	Descrizione / Simulazione o Fisico
cargorobot	Attivo (Software/Hardware)	Robot a guida differenziale (simulato o reale).
IOPort Pushbutton	Dispositivo di Input	Invia richieste asincrone a <code>cargoservice</code> .
IOPort Display	Dispositivo di Output	Aggiornato da <code>cargoservice</code> .
Sonar Sensor	Dispositivo di Input	Invia letture di distanza a <code>cargoservice</code> .
Marker	Dispositivo Attivo	Etichetta il container e invia il completamento.
Led	Dispositivo di Output	Componente software o hardware.

Metamodello di Comunicazione

- **Request/Reply:** Richiesta `load` del cliente, risposta del servizio. Comandi `move_robot` e relativi esiti.
- **Dispatch:** Eventi dal sonar (`sonar_data`, `sonar_fail`), dal marker (`marking_done`) e comandi display/led.

Architettura Logica Concettuale



Test plans

Caso di Test 1: Flusso Normale (Happy Path)

Stato Iniziale: Stiva vuota. Robot in HOME. Sonar ok.

Procedura: Richiesta accettata -> Container posizionato in tempo -> Spostamento slot5 -> Marcatura -> Spostamento slot finale -> Ritorno HOME.

Caso di Test 2: Rifiuto Stiva Piena

Stato Iniziale: Gli slot 1-4 sono tutti occupati.

Procedura: Richiesta inviata -> Sistema rifiuta la richiesta.

Caso di Test 3: Timeout del Cliente

Stato Iniziale: Stiva vuota. Robot in HOME.

Procedura: Richiesta accettata -> Container NON posizionato entro 30s -> Timeout -> Disimpegno e slot liberato.

Caso di Test 4: Guasto al Sonar (Fuori Servizio)

Procedura: Sonar invia D > DFREE per 3s -> Sistema va Out of service -> Display aggiornato -> Richieste respinte con `retrylater`.

Project

L'architettura logica QAK è disponibile in [cargoservice.qak](#) ([../src/cargoservice.qak](#)).

Obiettivo dello Sprint 1: Nello Sprint 1 implementeremo e raffineremo la logica centrale dell'attore **cargoservice**, realizzando i test automatizzati JUnit associati ai Test Plans qui sopra definiti.

Testing

(I test automatizzati JUnit saranno implementati nel prossimo sprint come definito nei Test Plans)

Deployment

(Sezione da compilare al termine degli sprint di sviluppo)

Maintenance

(N/A per Sprint 0)

Luigi Riccardo Simone  GIT repo:
<https://github.com/riccardocar99/cargoservice->